<u>Algorithms, AI, & ML Lab: Assignment 5</u>

Artificial Neural Network based Classifiers

1. **Prepare the dataset in desired representation:** Read the provided dataset. Name and detailed information about the features are given in "heart-disease.names" file, and the datasets to be used are in "processed.cleveland.data", "processed.hungarian.data", "processed.switzerland.data", and "processed.va.data". Prepare the data sample matrix *<u>Examples</u>*. Data curation should be generic enough to be applicable to any other randomly selected dataset provided in the similar format. Remove features "Slope" and "CA" from all datasets (as values are missing in many cases), and then consider only those examples without any missing data for the all below experimentation.
2. **Implement the Multiplayer Perceptron (MLP) Backpropagation learning algorithm**: Follow the generic Backpropagation algorithm as sketched in Table 4.2 (Page 98) of the Text-Book "Machine Learning" by Tom M. Mitchell (McGraw-Hill publications, 1997), and reproduced at the end of this assignment for ready reference. Provide adequate comments describing major steps of the algorithm to make it easy to read. Also, make sure that the developed code is generic enough with below hyper-parameters of the MLP model and Backpropagation algorithm can be adapted easily as per the need of the application.
    a. **Number of input nodes (*n_in*) and Number of output nodes (*n_out*)**: This should be chosen automatically from the *<u>Example</u>* data-matrix created in Step 1.
    b. **Number of hidden-layers (*num_hidden_layers*)**: Any user-defined positive integer value.
    c. **Number of nodes in each hidden layer (*n_hidden*)**: An array of length = num_hidden_layers and i^{th} element *n_hidden (i)* specifies the number of nodes in i^{th} hidden layer.
    d. **Learning Rate (η):** Any user-defined positive real number within [0, 1].
    e. **Learning Rate Decay Parameters (α, *LR_decay_freq*):** Decay rate <u>α</u> takes any user-defined positive real number within [0, 1] and applied after every *LR_decay_freq* (user defined any positive integer number) iterations to reduce the learning rate parameter as follows: [$\eta = \eta * \alpha$].
    f. **Termination conditions defined by Maximum number of iterations (*maxIter*), Maximum error tolerance (*maxErr*), and Percentage Overfitting (*percentOverfit*):** The hyperparameter *maxIter* (any user-defined large positive integer number) to be checked first to exit potential infinite loop after *maxIter* number of iterations. Then *maxErr* (any user-defined positive real number between [0, 100] defining percentage of total number misclassifications with respect to the total number of examples) to be checked for the ***<u>Test-Data only</u>*** to confirm convergence. And finally, *percentOverfit* defines the maximum limit on the difference between training-set accuracy and validation-set accuracy computed after every iteration.
    g. **Training:Validation:Testing dataset split ratio (defined by *Train_Seg*:*Val_Seg*:(100 – *Train_Seg* – *Val_Seg*)):** Train_Seg and Val_Seg are two user-defined positive integer values within [0, 100]. Dataset partition should be made randomly, without replacement.
3. **Use the developed MLP model to learn standard 2-bit XOR operation with the following setup**: [*num_hidden_layers* = 0], [η = 0.1], [α = 1], [*LR_decay_freq* = infinity], [Termination conditions setup as per your choice for acceptable convergence]. Use the entire data for training as well as testing.
    a. Plot the following variables over the iterations: (i) Total prediction error, (ii) values of the weights.
    b. Write the finally learned MLP in the form of equations of the form: y = sigmoid (sum (w_i * x_i)).
    c. Is there any effect on these plots and the learned parameters if the order of the examples changes?

d. If the same MLP model is trained for 2-bit XNOR operation, what will be the finally trained weight parameters? Compare these parameter values with the same obtained under 3b.

4. **Use the developed MLP model to learn an ANN classifier** for the Cleveland dataset in obtained in Step-1 with the following hyperparameters: **<u>Settings-1</u>**: [*num_hidden_layers* = 1], [*n_hidden* = [5]], [η = 0.1], [α = 1], [*LR_decay_freq* = infinity], [*maxIter* = 100], [*maxErr* = 5], [*percentOverfit* = 30], [*Train_Seg* = 80], [*Val_Seg* = 10]. Use standard Sigmoid activation in all the nodes. Consider fully connected layers – i.e., each node in current layer provides input to every node in the next layer. Initialize all the weight parameters to zero values ($w_{ij} = 0$ for all i, j). Consider first a binary classification – healthy (target feature = 0) vs. heart-disease (target feature > 0).
   - **a.** Plot the following for every iteration: (i) Training set error, (ii) Validation set error, (iii) Testing set error.
   - **b.** Find the Confusion matrix after every 20 iterations for the Testing data set.
   - **c.** Plot of the weight (2D) matrices as <u>*grayscale*</u> images, for the connections between every consecutive layer.
   - **d.** Find the confusion matrices when the model trained by Cleveland dataset is tested with other three (completely unseen) datasets obtained in Step 1.
   - **e.** Repeat Step 4d with the model being trained by other three datasets one at a time, tested by remaining three datasets.
   - **f.** Repeat Step 4a-d with model being trained by all four datasets considered at the same time.
   - **g.** Discuss on interesting patterns in the above results, if any.

5. **Experimentation with different variations as sketched below:** Repeat Step 4 with the following changes in the model structure and/or dataset.
   - **a. Increasing complexity of the model (Setting-2)**: Same as Setting-1, with number of hidden layers is changed to 2 with hidden nodes [5, 3].
   - **b. Increasing generalization (Setting-3)**: In Setting-1 above, change [training : validation : testing] split to [60 : 20 : 20].
   - **c. Effect of learning rate value towards convergence (Setting-4):** In Setting-1 above, change [α = 0.9], [*LR_decay_freq* = 20].
   - **d. Multiclass classification (Setting-5):** Now, in Setting-2, consider all heart-disease classes separately. Compare the results with 5a above, and discuss on the accuracy for different classes and different datasets considered separately or jointly.

6. **Handling Data with Missing Feature Values:** Repeat Step 5 with replacing the missing features values by a simple sub-optimal estimation method as described below. And then compare the results obtained with using datasets "without-missing-data-replacement" and "with-missing-data-replacement".
   - **a.** For any particular data example $\mathbf{x_m}$ with a missing feature value (x_b), find-out the target class ω_i for that data example, find all data examples (D_1) with same target-class ω_i with **<u>no</u>** missing data.
   - **b.** For D_1 subset of clean data, consider all other features (except x_b), and find out of them one feature (x_g) with the highest correlation value between x_g and x_b (or pair-wise similarity based on 2-dimensional joint histogram for discrete features x_g and x_b). Develop a 2D scatter plot (or 2D histogram, if discrete features) with (x_g and x_b) in two axes.
   - **c.** Now, find the value of x_g for $\mathbf{x_m}$, which is present and correct. Use this value to find out the highest density of the scatter plot (or histogram) to estimate the missing value x_b for $\mathbf{x_m}$.
   - **d.** For data-samples with multiple missing features, repeat the above steps 6a-d multiple times to replace the missing values.

7. Next.

BACKPROPAGATION($training_examples, \eta, n_{in}, n_{out}, n_{hidden}$)

Each training example is a pair of the form $\langle \vec{x}, \vec{t} \rangle$, where $\vec{x}$ is the vector of network input values, and $\vec{t}$ is the vector of target network output values.

η is the learning rate (e.g., .05). n_{in} is the number of network inputs, n_{hidden} the number of units in the hidden layer, and n_{out} the number of output units.

The input from unit i into unit j is denoted x_{ji}, and the weight from unit i to unit j is denoted w_{ji}.

- Create a feed-forward network with n_{in} inputs, n_{hidden} hidden units, and n_{out} output units.
- Initialize all network weights to small random numbers (e.g., between $-.05$ and .05).
- Until the termination condition is met, Do
  - For each $\langle \vec{x}, \vec{t} \rangle$ in $training_examples$, Do

    Propagate the input forward through the network:

    1. Input the instance $\vec{x}$ to the network and compute the output o_u of every unit u in the network.

    Propagate the errors backward through the network:

    2. For each network output unit k, calculate its error term δ_k

$$\delta_k \leftarrow o_k(1 - o_k)(t_k - o_k) \tag{T4.3}$$

    3. For each hidden unit h, calculate its error term δ_h

$$\delta_h \leftarrow o_h(1 - o_h) \sum_{k \in outputs} w_{kh}\delta_k \tag{T4.4}$$

    4. Update each network weight w_{ji}

$$w_{ji} \leftarrow w_{ji} + \Delta w_{ji}$$

    where

$$\Delta w_{ji} = \eta\, \delta_j\, x_{ji} \tag{T4.5}$$